LAB 2 | End-to-End E-Commerce Order Processing

ABHINAV L | RA2411003010553

Course: WORKFLOW MANAGEMENT

Camunda 8.8 | Java 17 | Spring Boot 3.5.6 | Synthetic data

Aim

Validate an order, calculate its total and stock status, determine delivery priority, review high-value orders, and simulate payment and fulfilment.

Architecture and execution

1. The starter receives customerRef, productId, quantity, deliveryPreference, customerCategory and a synthetic paymentToken.

2. The order-validate worker verifies required fields, supported values and integer quantity 1..1000. It uses a fixed catalog to calculate orderTotal and stockAvailable.

3. An invalid order stops before payment. Insufficient stock ends as Out of stock. Valid in-stock orders evaluate OrderPriority.

4. The FIRST DMN table returns High, Medium or Normal deliveryPriority. This is separate from the manual-review rule.

5. Orders with orderTotal >= 50000 enter a Tasklist verification task. Staff approval continues to payment; rejection finishes without charging.

6. The REST connector calls /mock/payment. Paid proceeds to simulated fulfilment; Declined or another non-paid business response ends as Payment declined.

7. HTTP 503 creates a connector incident. A recoverable inventory fault occurs in the worker. Clear the fault and retry the SAME process instance.

Rules, samples and assumptions

Hit policy: FIRST. The first matching rule returns one output.

Order	Condition	Output
1	deliveryPreference = Express	High
2	customerCategory = VIP	High
3	orderTotal >= 50000	High
4	orderTotal >= 10000	Medium
5	Otherwise	Normal

Total; customer category; delivery preference	Expected result
1000; Standard; Standard	Normal
1000; VIP; Standard	High
1000; Standard; Express	High
10000; Standard; Standard	Medium
50000; Standard; Standard	High

Validation

The existing local suite passed 39 JUnit tests, including 19 standalone DMN evaluations, plus 20 mock HTTP checks. Standalone rule tests used Camunda 7.24; live workflow screenshots separately demonstrate Camunda 8 process execution. Logs are retained under evidence/.

Assumptions

Inventory is a fixed mock snapshot, not a reservation system. Fulfilled is a simulated outcome, not a real shipment. No money is charged. Payment responses are cached by caseId in memory and reset on restart. Separate cases require unique caseIds. High priority does not itself imply manual review; the review threshold is total >= 50000.

Configuration and variables

README.md supplies startup/deployment commands. API_EXAMPLES.md documents REST mappings and request/response bodies. VARIABLE_DICTIONARY.md defines variable names, types and purposes. Editable models and all source code are included.

Observed runtime results

Live execution was performed on a Windows/Docker installation on 30-31 October 2026. Results below were transcribed from Operate/Tasklist screenshots and copied process variables. Fifteen required scenarios have observed terminal outcomes. Seven additional example scenarios are not claimed as live-tested. Screenshot times are shown as displayed on the test PC.

Case / input JSON	Expected	Observed / instance	Status
O1-paid.json	Fulfilled	Paid; Fulfilled 2251799813696379	Passed
O2-no-stock.json	Out of stock	Out of stock 2251799813702359	Passed
O3-declined.json	Payment declined	Declined; Payment declined 2251799813702757	Passed
O4-high-value.json	High-value review; Fulfilled after reviewer approval	High-value review -> approved -> fulfilled route completed 2251799813704225	Passed
O5-payment-error.json	Payment connector incident; Fulfilled after fault cleared and retry	Payment incident -> correction/retry -> fulfilled route completed 2251799813715498	Passed

Additional examples

O6-inventory-error, O7-review-reject, O8-invalid: supplied but not live-tested.

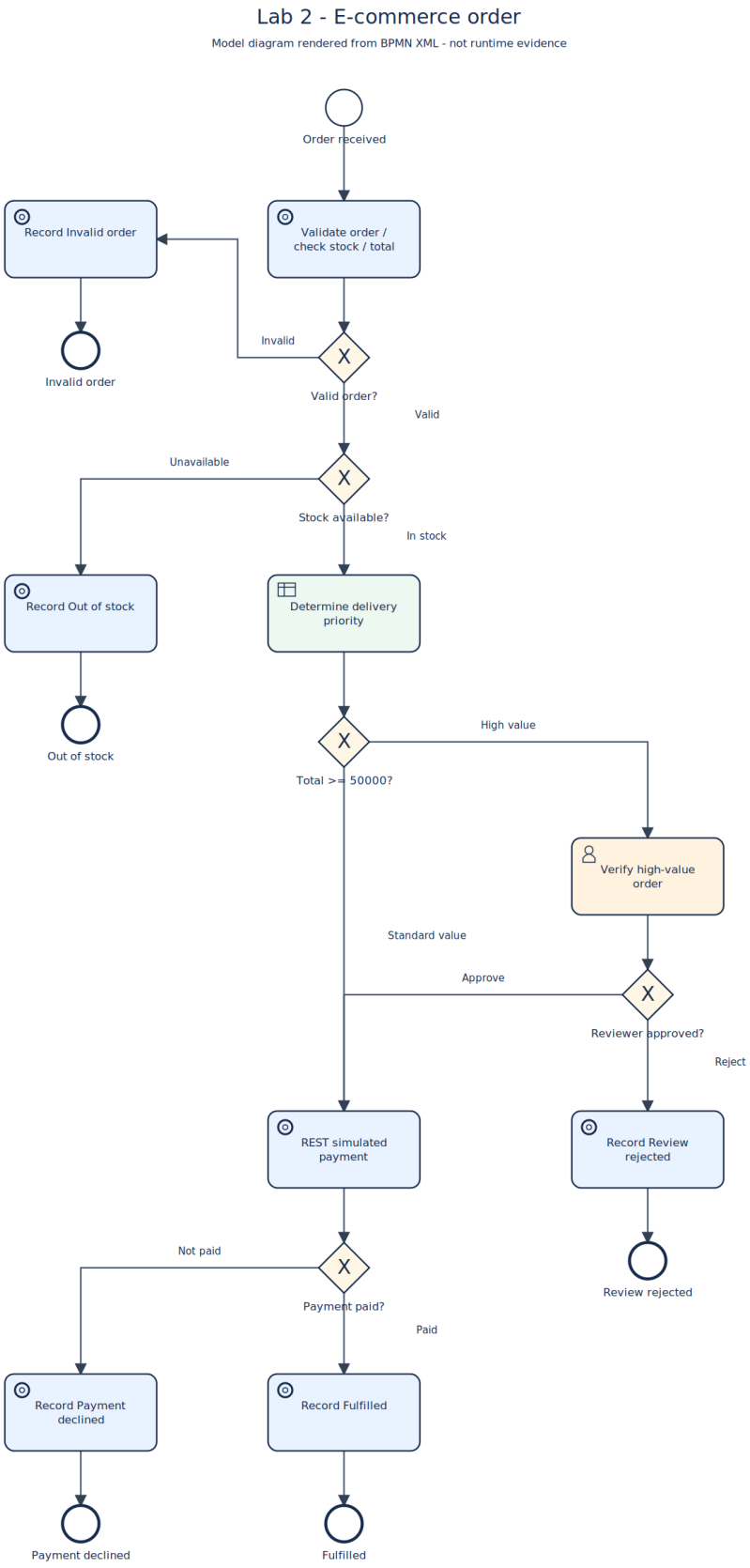
Evidence boundaries

O5 incident and completion screenshots show the same instance key. O4 completed history demonstrates the high-value user-task route; the earlier form image is pre-submission evidence.

Recovery procedure

Enable the case-specific mock fault, start the example and inspect the incident in Operate. Clear that fault and retry the failed job in the same instance. The failed external service returns HTTP 503; worker/connector failure exhausts retries and creates an incident. Loan worker faults are injected in validation. H5 reaches the scheduling API first through the availability worker. No real services or payments are involved.

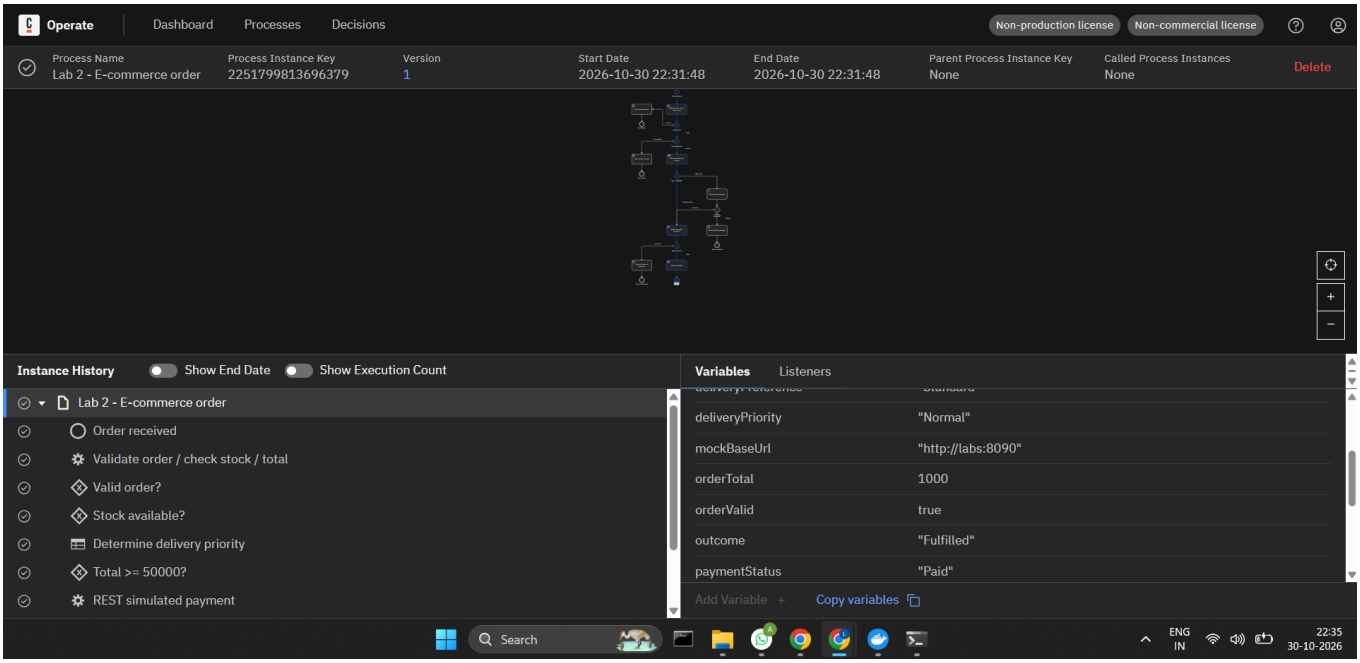
BPMN process design



Rendered from the supplied BPMN XML. Runtime captures follow; full-resolution original screenshots are in evidence/runtime/.

O1-paid

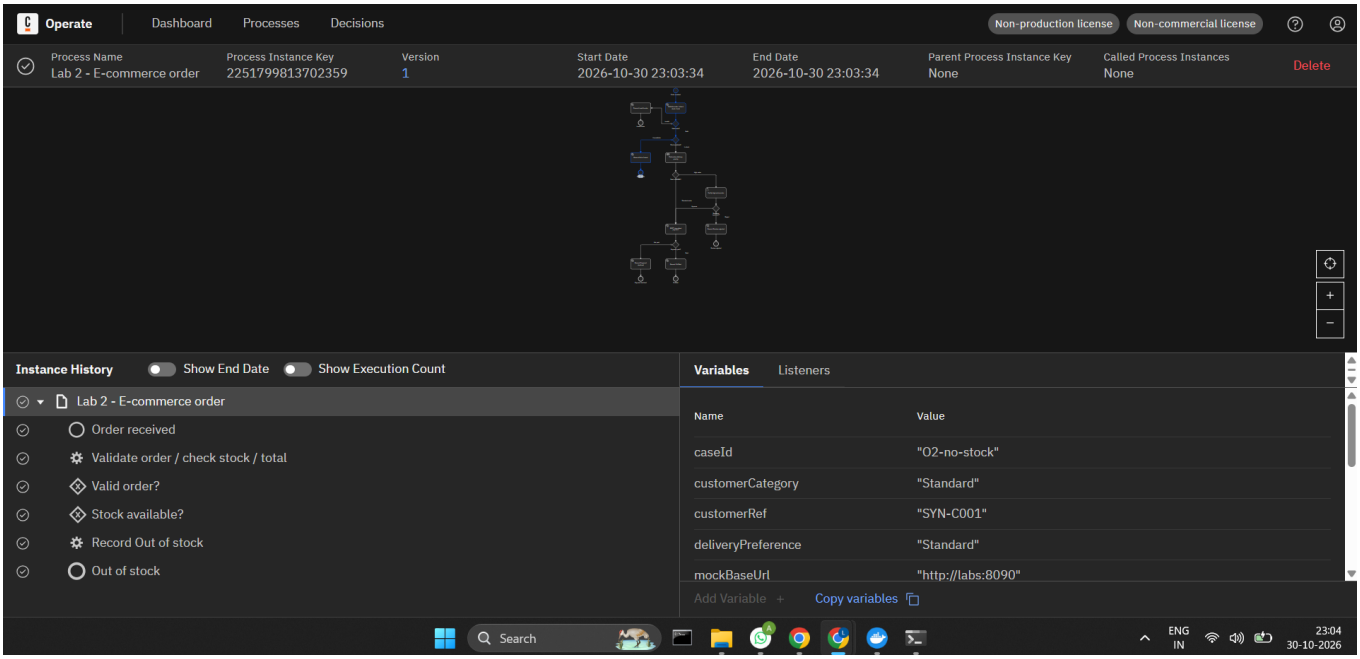
Paid and Fulfilled variables in completed instance.



Full-resolution evidence: evidence/runtime/O1-paid.png

O2-no-stock

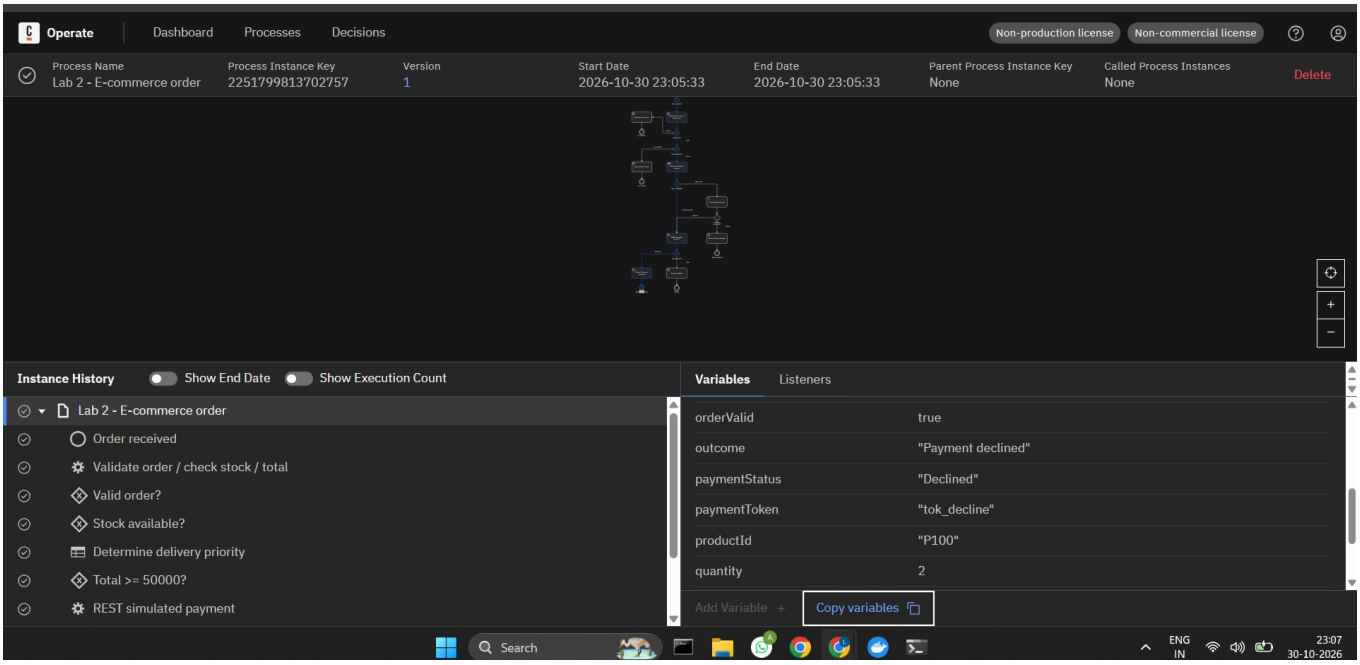
Completed Out of stock path.



Full-resolution evidence: evidence/runtime/O2-no-stock.png

O3-declined

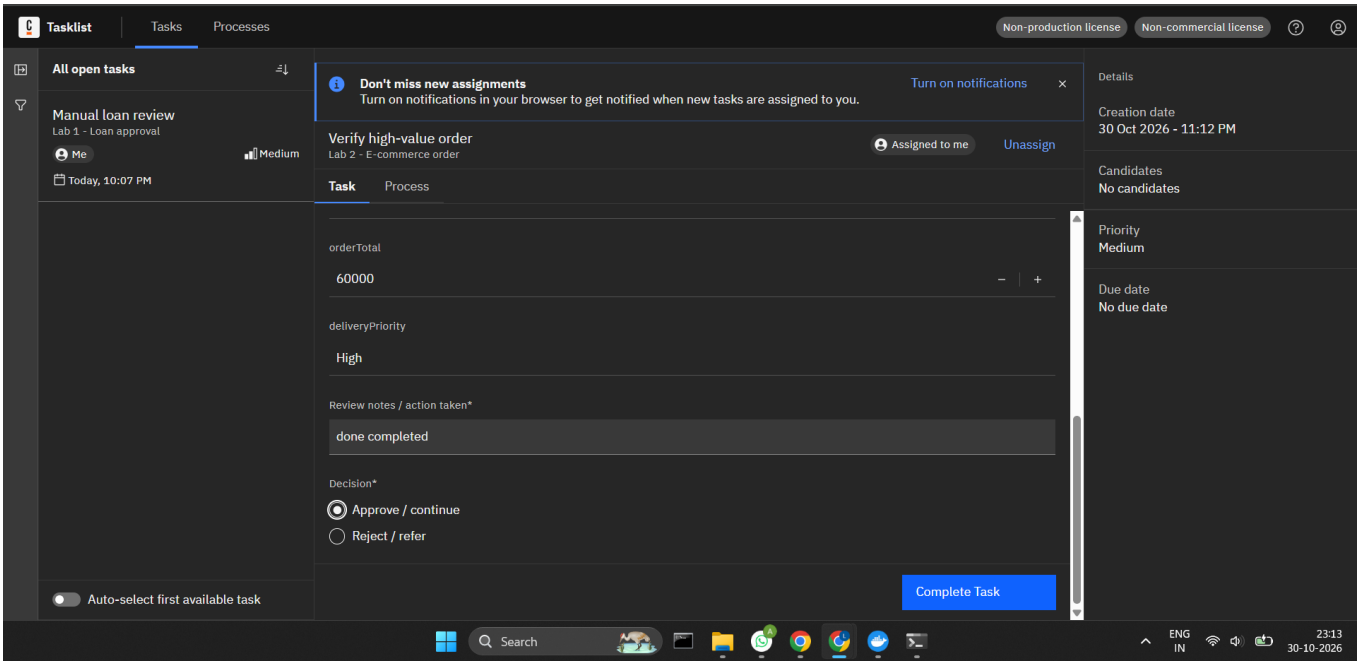
Payment declined outcome and Declined status.



Full-resolution evidence: evidence/runtime/O3-declined.png

O4-review

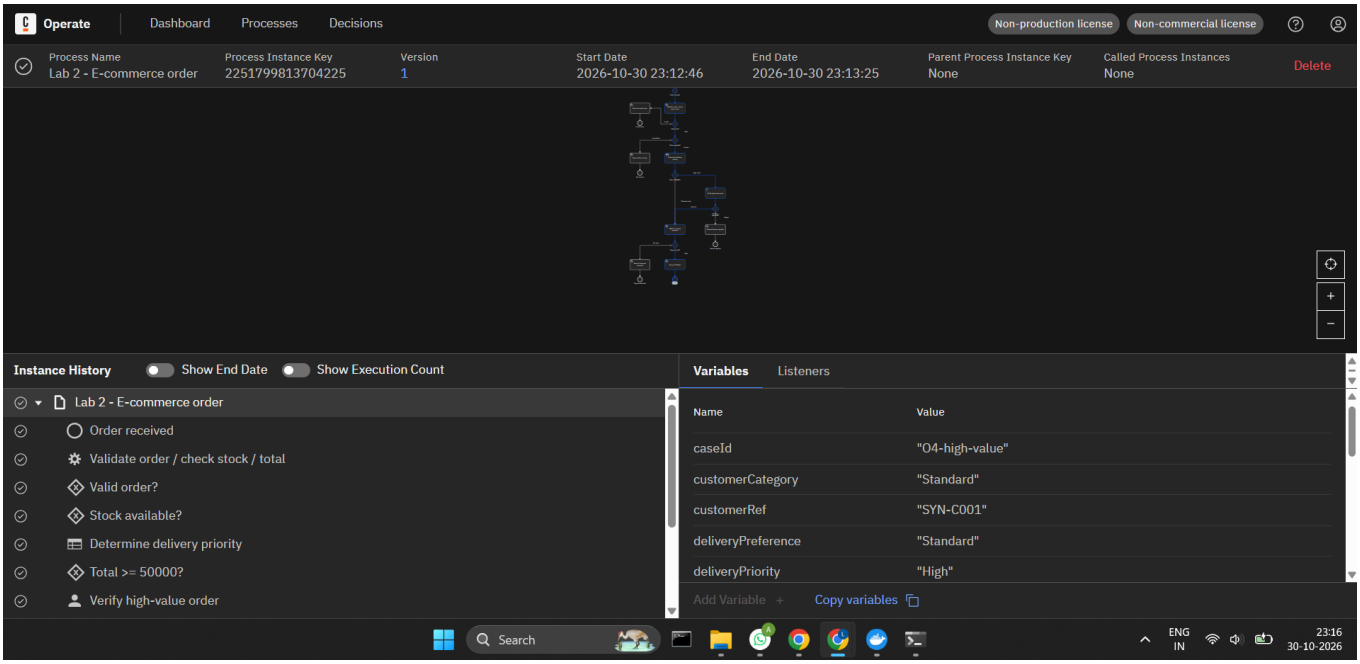
High-value order form: total 60000, High priority and Approve selected. Original upload filename incorrectly said Lab3.



Full-resolution evidence: evidence/runtime/O4-review.png

O4-completed

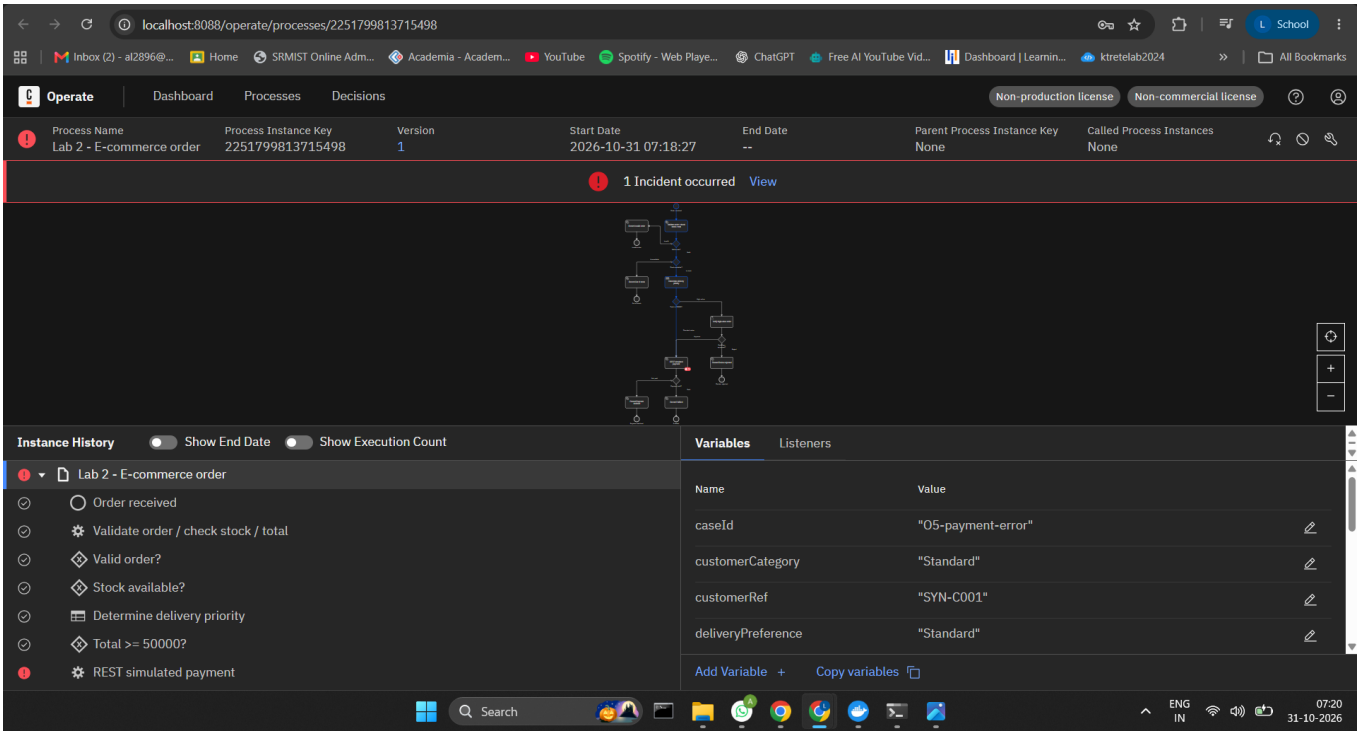
Completed high-value review and fulfilment path.



Full-resolution evidence: evidence/runtime/O4-completed.png

O5-incident

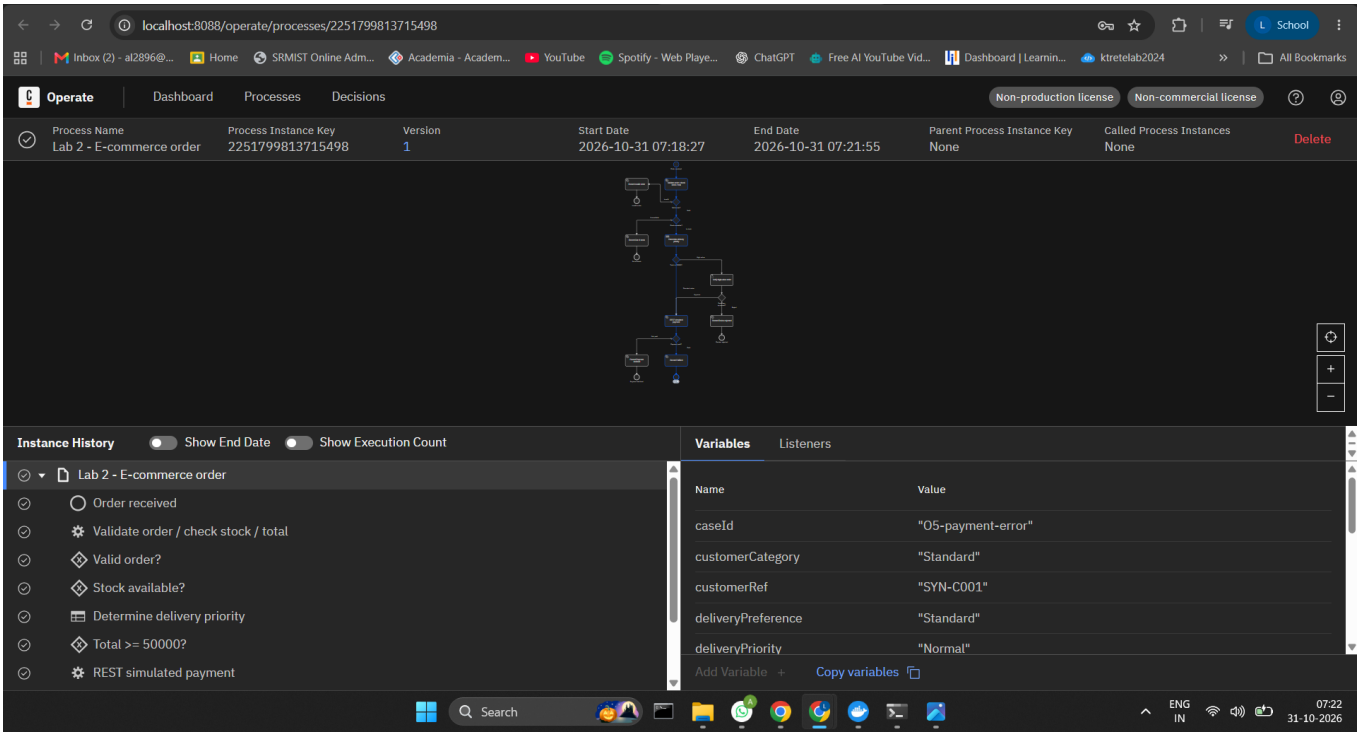
Payment connector incident in instance 2251799813715498.



Full-resolution evidence: evidence/runtime/O5-incident.png

O5-recovered

Same payment-error instance completed after correction and retry.



Full-resolution evidence: evidence/runtime/O5-recovered.png